

real_vpn_egress_proof.sh — companion guide (§11.4.18)

Revision: 1 **Last modified:** 2026-07-01T06:40:00Z **Script:** tests/egress_proof/real_vpn_egress_proof.sh **Workable item:** #54 — real-VPN-egress functional proof (the POSITIVE egress half)

Overview

real_vpn_egress_proof.sh is the **creds-drop-ready** harness that proves the *positive* half of the dynamic VPN-aware proxy: that with **real** operator-provisioned gluetun WireGuard credentials, a real packet leaves through the tunnel and the egress public IP observed **through the proxy** equals the tunnel exit **and** differs from the host's own IP (design §15 / research §15 — the hardest-to-fake routing proof).

The complementary *negative* half — **fail-closed security** (tunnel-down → branded 503, no leak) — is already proven, independently, by the committed Go integration test control-plane/cmd/healthd/healthd_integration_test.go (TestIntegration_HealthdWritesDownAgainstRealGluetun, assertion at :174-176): a REAL gluetun booted with a FAKE WireGuard config produces an EMPTY egress, so healthd MUST write a DOWN snapshot. This harness does **not** re-prove that; it targets only the egress PASS that needs real key material.

Real WireGuard keys are **operator-provisioned** (§11.4.21 / §11.4.52) and are **never** fabricated, invented, or committed (§11.4.10). The harness therefore has two honest outcomes and never fakes a pass.

Prerequisites

Requirement	Why
Operator WireGuard creds in gitignored ./env (or exported)	Only the creds path boots the stack; absence → honest SKIP.
Rootless podman on PATH (§11.4.161)	Runs the tunnel-UP probe (podman exec proxy-gluetun ...).

Requirement	Why
curl, awk, grep, ss/netstat	Egress capture + the §11.4.174 port guard.
The sanctioned ./start / ./stop orchestrators	Boot/teardown — never raw podman run.
A free :53128 (HTTP_PROXY_PORT) + no running proxy-squid/proxy-gluetun	Guarded; the harness refuses (exit 3) rather than disturb a foreign owner.

The five operator cred vars (the **exact** names the dynamic overlay reads — docker-compose.dynamic.yml:149-153, .env.example:177-182):

```
WIREGUARD_PRIVATE_KEY  WIREGUARD_PUBLIC_KEY  WIREGUARD_ADDRESSES
WIREGUARD_ENDPOINT_IP  WIREGUARD_ENDPOINT_PORT
```

Usage examples

```
# Auto-detect creds. No creds -> clean SKIP (exit 0). Creds ->
  real-egress PASS.
```

```
tests/egress_proof/real_vpn_egress_proof.sh
```

```
# Optional overrides:
```

```
EXPECTED_EXIT_IP=185.65.135.70 \
```

```
PROXY_PORT=53128 \
```

```
IP_ECHO_URL=https://icanhazip.com \
```

```
BOOT_TIMEOUT=180 \
```

```
  tests/egress_proof/real_vpn_egress_proof.sh
```

Outcomes:

Condition	stdout verdict	Exit
Creds absent	SKIP: ... [reason: operator_attended]	0
Creds present, egress == exit && != host	PASS: ... [evidence: .../egress_via_proxy.ip]	0
Creds present, egress == host / wrong exit / no egress	FAIL: ... [reason: ...]	1
:PROXY_PORT bound OR a proxy-* container already up	FAIL-SAFE ... NOT booting (§11.4.174)	3
No container runtime	SKIP: ... [reason: hardware_not_present]	0

Edge cases

- **No .env at all** → treated as creds-absent → SKIP (the default developer state).
- **Partial creds** (any one of the five empty) → creds-absent → SKIP (never a half-boot).
- **Secrets never printed** — presence is tested via `grep -q exit-status / a length test`; the key value is never echoed or logged (§11.4.10). Safe to run with verbose shells.
- **Foreign proxy stack already up** (e.g. proxy-squid from another session on the shared host) → exit 3, nothing touched (§11.4.174). Observed live during authoring: a pre-existing proxy-squid "Up 5 hours" holding `:53128` correctly caused the guard to refuse.
- **Tunnel never comes UP within BOOT_TIMEOUT** → FAIL (bad creds/endpoint), stack torn down by the trap.
- **Re-runnable / deterministic** (§11.4.98): SKIP path returns exit 0 identically across repeated runs; no manual step after start.

Internal behaviour

1. Source the committed `tests/lib/evidence.sh` (`ab_skip_with_reason`, `ab_pass_with_evidence`, `assert_egress_ip` — the §15 bluff-catcher, self-tested).
2. Detect the five cred vars (`env` **or** `.env`) without capturing/printing any value.
3. **Absent** → `ab_skip_with_reason ... operator_attended`, write `verdict.txt`, exit 0.
4. **Present** → detect runtime; §11.4.174 guard on `:PROXY_PORT + proxy-*` names; if contended, exit 3 **without booting**.
5. `./start --dynamic` (sets `BOOTED=1`); poll `gluetun /v1/publicip/ip` until non-empty.
6. Capture egress-via-proxy (`curl -x`) + host-direct IP into `qa-results/issue54/`; delegate the verdict to `assert_egress_ip` (`egress == exit AND != host`).
7. trap cleanup EXIT runs `./stop` iff `BOOTED=1` — quiescent on every path (§11.4.14).

Evidence artefacts (under `qa-results/issue54/`, gitignored raw corpus §11.4.30): `verdict.txt`, `egress_via_proxy.ip`, `host_public.ip`, `expected_exit.ip`.

Related scripts

- `tests/lib/evidence.sh` — the sourced §11.4.69 evidence helpers (`assert_egress_ip:213`).

- control-plane/cmd/healthd/healthd_integration_test.go — the fail-closed (security) proof.
- tests/dynamic/analyzers/egress_neq_host_analyzer.sh — the offline golden-good/bad analyzer for the same egress-≠-host oracle.
- ./start / ./stop — the sanctioned dynamic-stack orchestrators.
- docs/DYNAMIC_VPN_EGRESS_PROOF.md — the operator runbook (how to supply creds + run).

Last verified

2026-07-01 — SKIP path executed (exit 0, deterministic ×3);
creds-present branch + §11.4.174 port guard executed (exit 3, did not boot); sh -n + bash -n parse-clean.